

# **Individual Project Final Report**

ID Number: F213049

Programme: Robotics, Mechatronics and Control Engineering

Module Code: 25WS50META

Project Title: Project Grandmaster: An Autonomous Robotic Opponent for Go.

## **Abstract:**

This report presents the design and implementation of an autonomous robotic system capable of acting as a partial physical opponent for the board game Go. The primary aim was to develop a 3 degree of freedom SCARA based robotic arm capable of reliably performing pick and place operations of Go stones on a Go board.

The system was brought to life through a combination of mechanical design, hardware implementation and software control. A SCARA configuration was selected for its suitability in such tasks, and the robot was made to meet workspace and precision requirements. Forward and inverse kinematics were explored to compute joint angles from board coordinates. Control was achieved using a micro controller, with formatted commands converted from the user input by a Python Graphical User Interface.

The robot demonstrated reliable operation across the defined workspace. It achieved consistent pick and place operations with the gripper, reaching 100% success rate under specific conditions. With an efficient inverse kinematics algorithm, the system maintained good repeatability throughout experiments. However, the system suffered minor precision errors due to mechanical backlash. Additional limitations included lack of feedback control and flexibility of the device.

In conclusion, the project achieved its objective of developing a functional robotic Go opponent at a low-level. While the output has successfully met most objectives, the integration of computer vision and an AI Go engine is required to achieve full autonomy.

# 1 Table of Contents

|       |                                   |    |
|-------|-----------------------------------|----|
| 2     | Introduction .....                | 3  |
| 2.1   | Project background .....          | 3  |
| 2.2   | Project context .....             | 3  |
| 2.3   | Project aims and objectives ..... | 3  |
| 2.4   | Report description .....          | 4  |
| 3     | Literature review.....            | 4  |
| 3.1   | Related work .....                | 4  |
| 3.2   | Robot architecture .....          | 5  |
| 4     | Methodology.....                  | 6  |
| 4.1   | Model .....                       | 6  |
| 4.1.1 | Constraints .....                 | 6  |
| 4.1.2 | Structure.....                    | 6  |
| 4.1.3 | Links and joints.....             | 6  |
| 4.1.4 | Kinematics .....                  | 8  |
| 4.1.5 | Inverse Kinematics .....          | 9  |
| 4.2   | Hardware.....                     | 9  |
| 4.2.1 | Links CAD and 3d printing .....   | 9  |
| 4.2.2 | Actuation system.....             | 10 |
| 4.2.3 | Transmission mechanism .....      | 11 |
| 4.2.4 | End-Effector design .....         | 11 |
| 4.2.5 | Power supply .....                | 12 |
| 4.2.6 | Control Hardware.....             | 12 |
| 4.2.7 | Circuit and assembly .....        | 13 |
| 4.3   | Software .....                    | 14 |
| 4.3.1 | Arduino IDE .....                 | 14 |
| 4.3.2 | System setup .....                | 14 |
| 4.3.3 | Target Coordinates.....           | 15 |
| 4.3.4 | Sequences .....                   | 15 |
| 4.3.5 | Serial commands .....             | 16 |
| 4.3.6 | Playing Interface.....            | 16 |
| 4.3.7 | Workflow.....                     | 17 |
| 5     | Conclusion .....                  | 18 |
| 5.1   | Results .....                     | 18 |

|     |                       |    |
|-----|-----------------------|----|
| 5.2 | Limitations.....      | 18 |
| 5.3 | Discussion.....       | 19 |
| 6   | Acknowledgements..... | 19 |
| 7   | References.....       | 19 |
| 8   | Appendix .....        | 21 |

## 2 Introduction

### 2.1 Project background

Amongst many other board games, the game of Go stands out for its ancient origins. Go, as it is known in Japan, is an ancient Chinese board game conceived more than 2,500 years ago [1], and it is believed to be the oldest board game that is still played today. Also known as *Weiqi* (圍棋) in China meaning "surrounding game", the goal is to capture more territory than your opponent. The players do so each in turn by placing stones on the checkered lines of the board. Although, the board can vary in size depending on the level of the player, the game is commonly played on a 19x19 board. It has very simple rules and concepts, and yet it is considered by many people to be the world's greatest strategic game, far surpassing Chess in its complexity [2]. It was brought to the UK from Japan over a century ago and has since gained in popularity.

### 2.2 Project context

Developments in technology have progressively shifted board games, such as Go, on digital platforms. Modern video games offer many advantages over board games. Players can instantly access a wide variety of games at any time and connect with opponents globally without the constraints of location or time. However, there has recently been renewed interest in physical board games. This resurgence in popularity has led to their increasing commercial success [3]. Despite this rising phenomenon, there remains a paucity of evidence on the reasons why players would rather choose board games over online ones. It is important to note that the relative preference of board gaming is debated in the literature. It can be argued that playing on a physical board offers a tactile and sensory experience sought by players. Recent evidence shows that the materiality provided by physical components is a key element in the player experience that digital formats cannot replicate [4]. On the other hand, board games also present serious drawbacks. The main limitation lies in requiring another player. Players often find themselves unable to engage with board games particularly when no suitable opponent is available. This creates a challenge, leading many to seek a form of the game that combines the advantages of both physical and digital versions. Restoring the authenticity of board games whilst no longer relying on the presence of a fellow player. A solution that can provide a constant, capable and physical opponent allowing the gaming experience to be enjoyed to its fullest.

### 2.3 Project aims and objectives

Drawing upon the challenges associated with board games, this report aims to build an autonomous robot that can master the ancient game of Go. This report focuses on designing, building, and

validating a 3-DOF mechatronic robotic arm pick-and-place subsystem capable of reliably moving standard Go stones between specified board locations, using a defined coordinate interface and motion control. It is beyond the scope of study to encompass the implementation of an intelligent computer vision system with a powerful, pre-existing AI game engine.

The main objectives of the project were to:

1. Design and build a complete 3-DOF robotic arm electro-mechanical assembly, including CAD, actuator selection, mounting and joint structures capable of reaching all positions within the defined planar workspace.
2. Design and build a Go-stone gripper including its actuator, mounting interface to the arm, and required electrical connections capable of reliably picking up and placing Go stones.
3. Implement inverse kinematics to convert board coordinates into joint angles, enabling the end-effector to reach target positions within a positional accuracy of approximately 2 mm.
4. Develop and implement an interactive graphical user interface that converts user input into structured commands, transmitted to the motion controller under a second for execution in real-time.

## 2.4 Report description

This report begins with a brief literature review, examining related work in the field and possible robot architectures fit for this project. It then proceeds to the main body, which is organised into four chapters. The first section of this report will examine the model of the system including design constraints, overall structure, links and joints, kinematic model, and inverse kinematics of the robot. The second part will be concerned with the hardware of the project. Specifically, the selection and development of key components such as the CAD links, actuation system, transmission mechanism, end-effector, power supply, and control hardware. Chapter three will begin laying out the implementation of the control system, including the Arduino setup, and it looks at how the target coordinates are mapped, movement sequences operated and serial command handled. This is followed by the description of the Python-based playing interface as well as the workflow of the overall system. The last section will present the results of the project, focusing on the limitations and considering the possible future developments of the project.

## 3 Literature review

### 3.1 Related work

Whilst some research has been carried out on other board game playing robots, only a limited number of Go playing robots have been identified. One of which is a commercialised device, called senseRobot [5]. It features an AI engine integrated in a robotic arm, which provides a learning platform for beginners and children. The robot also covers a wide variety of other board games such as Gomoku, Xiangqi and chess. Similarly, a great deal of previous research into board game playing robots has focused on chess [6,7,8,9,10,11]. A number of these studies focus on the implementation of a chess engine with a robot, that has not been designed for chess, to offer a physical opponent. In order to deliver the task some of the researchers have opted to work with either a Cartesian robot, a custom 6 DOF robot, or a collaborative UR5 robot. Despite having different mechanical structures, these systems share the same components, automatic detection and recognition of the board enabled with a vision system and learning algorithms. Such approaches are relevant because

chess, like Go, is an ancient board game played on a wood board around the same size. However, these studies have failed to address the inclusion of a Go stone specific end-effector and the implementation of a Go engine in a manipulator. A search of the literature revealed studies which cover Go engines alone [12,13]. The algorithms use reinforcement learning to play against opponents and are deployed on board games interfaces when facing a physical player. Their performance is unmatched compared to human's success rate and reach levels far beyond worldwide professional players. But up to now, far too little attention has been paid to implement these same AI agents into a physical game-playing robot. Many hobbyists have attempted to bridge the gap however none of them qualify as academic peer reviewed papers and therefore this idea has received scant attention in the research literature.

### 3.2 Robot architecture

Given the lack of groundwork for Go playing robots, plenty of freedom is given with regards to the mechanical design. From a mechanical perspective, the system can be fundamentally characterized as a pick and place device. Research has been led to better understand the mechanisms of different robotic structures and their underlying advantages depending on the required task [14,15,16]. These studies offer a classification of robotic architecture and their uses. They suggest that there are five main robot types: cylindrical, articulated, SCARA, Cartesian and Delta robots among many others. Out of which, three configurations emerge from the studies mentioned so far used to perform pick and place operations: specifically, the SCARA, Delta and Cartesian robots. There seems to be some evidence to indicate they supply high speed and good repeatability making them ideal for such tasks. A number of studies have postulated that Delta robots despite having low payload capacities and a small workspace, have extremely high-performance speed and precision [17,18,19]. These aspects theoretically lend themselves well for picking and dropping stones on a board. However, their parallel-link structure introduces significantly greater kinematic complexity compared to other configurations, making both implementation and control more challenging. In contrast, other writers have adopted Cartesian architectures instead [20,21,22]. Together these studies provide important insights into the benefits and downsides of this configuration. Cartesian robots have proven to be equally good in both speed and precision. Additionally, it is noted that their rigid axis allows them to sustain much higher loads. However, this comes at the price of size, which impairs the workspace and in turn limits the reach of the end-effector. Although the dynamics fit the gameboard narrative well, in a Go game setting, the rigid axes might interfere with the human opponent's personal space while playing. Along the same lines, many recent studies have also shown that SCARA robots are ideal for pick and place purposes [23,24,25]. Their work demonstrates that this configuration is consistent with previous ones, showcasing high speed and precision especially for planar tasks, with a moderate payload capacity. Such characteristics make them highly effective for Go-based tasks, which are performed over a flat even surface. However, there are noted flaws in the vertical axis, having limited reach and low flexibility. But these constraints are acceptable for the current application, where motion is largely planar and height variation is minimal. Overall, SCARA robots provide a more compact and accessible design than Cartesian systems, and a simpler, more practical implementation compared to Delta configurations. In all the studies reviewed here, precision is recognised as the key indicator for pick and place tasks. Given all the design trade-offs, cost and existing literature on Go playing robots, considerable evidence has accumulated to show that SCARA robot structures are best suited for this project.

## 4 Methodology

### 4.1 Model

#### 4.1.1 Constraints

To inform the design of the SCARA robot, the physical dimensions of the board and stones were identified, as these defined the system requirements. It should be noted that, for the purposes of this project, Go related materials were homemade and therefore did not fully reflect traditional designs. This project is focused on the second smallest Go board size, a 5x5 Go board. Each twenty-five playable position on the board was an intersection of a grid formed by a windowpane pattern. The board dimensions were as follows 130 mm in width and length, with row spacing of 23 mm and column spacing of 22 mm, resulting in a slight asymmetry. The Go stones used in this project have a diameter of 18.0 mm and are 1mm thick, resulting in a negligible mass. This is important because weight of the Go stones and their dimension influenced directly the design of robot and its the end-effector. Additionally, stones were stored in bowls during gameplay, which has been considered in this design.

Based on these constraints, the manipulator was required to have a minimum reach of at least 135 mm to fully cover the board. Furthermore, the robot was required to achieve a precision  $\pm 2$  mm to accurately pick and drop stones at specific intersections in order to avoid overlapping stones. Finally, it was required to have a minimum vertical range of 5 mm to allow for clearance above board and stones during manipulation, meanwhile payload requirements remained minimal.

#### 4.1.2 Structure

To cater for all constraints, the adopted design conformed to a SCARA-based structure with an open kinematic configuration. The robotic arm consisted of a series of links connected by three joints, forming an open-kinematic chain. One end of chain was connected to the base, while the other end was equipped of an end -effector to manipulate Go stones [Fig 1]. The system followed an RRP joint configuration which provided three degrees of freedom. The two first joints were revolutes, enabling rotational motion, while the third one was prismatic, providing linear motion [Fig 2]. This arrangement enabled planar motion in the horizontal plane combined with vertical motion for picking and dropping stones.

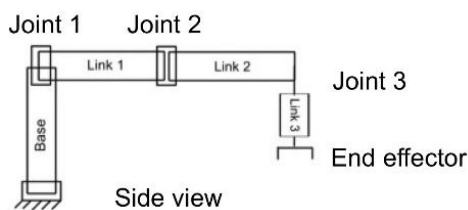


Figure 1: Scara robot configuration

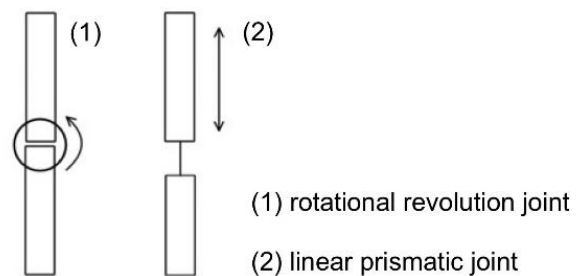


Figure 2: Joint configurations

#### 4.1.3 Links and joints

The manipulator workspace was determined by the number of joints, their geometric arrangement, and the link dimensions. In this design, the link lengths were selected to ensure that the robot can

cover the full board. The combined lengths of the two first links allowed the wrist frame to reach all twenty-five intersections. Additional length in the links was considered for the robot base to be positioned at an offset from the board rather than directly adjacent to it. Overall, the link lengths were chosen to maximize reachable workspace while keeping the structure compact and reducing the torque required for each revolute joint.

Table 1. Dimensions

| Links | Lengths | Units       | Joints | Range | Units       |
|-------|---------|-------------|--------|-------|-------------|
| 1     | 110     | Millimetres | 1      | 270   | Degrees     |
| 2     | 90      | Millimetres | 2      | 270   | Degrees     |
| 3     | 15      | Millimetres | 3      | 5     | Millimetres |

For the first two links, the planar RR reachable workspace, where  $r$  the distance from the base to the end-effector, was:

$$|L_1 - L_2| \leq r \leq L_1 + L_2 \quad 20 \leq r \leq 200 \text{ mm}$$

The model reached points at distances between 20 mm and 200 mm from the base. Only the offset between the base and the board would determine whether the intersections are within the reachable workspace. As it were, the base frame of the robot stood 140mm away from the centre of the Go board. As a result, any desired target point on the board lied within the reachable workspace of the manipulator. To maximise the workspace, each revolute joint was designed with a rotational range of  $270^\circ$  [Figure 3], allowing the manipulator to access a wider set of positions such as the stones contained in the bowls.

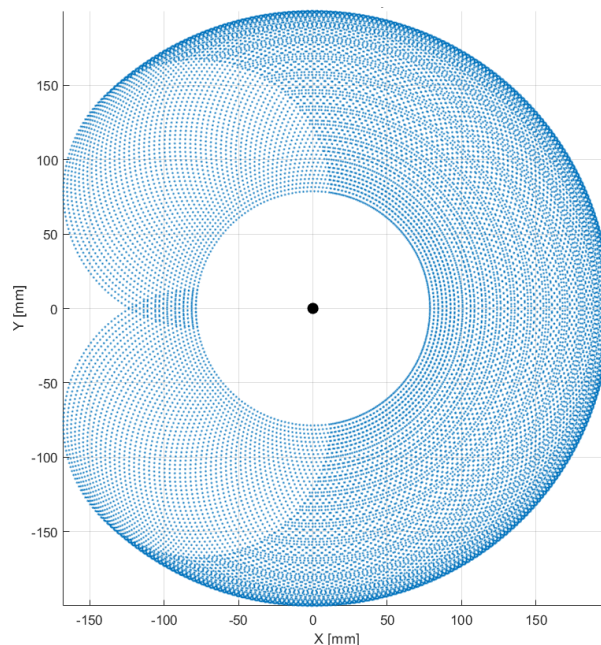


Figure 3: Reachable workspace

#### 4.1.4 Kinematics

Given a set of joint angles, the position and orientation of the tool frame relative to the base frame of the manipulator were computed using forward kinematics. The composition of the link parameters of the open-chain manipulator was as follows:

Table 2. DH parameters of SCARA robot

| Link number | Link length [mm] | Twist angles   | Link offset [mm] | Joint angles   |
|-------------|------------------|----------------|------------------|----------------|
| i           | a <sub>i</sub>   | α <sub>i</sub> | d <sub>i</sub>   | Θ <sub>i</sub> |
| 1           | 110              | 0              | 50               | Θ <sub>1</sub> |
| 2           | 90               | 0              | 0                | Θ <sub>2</sub> |
| 3           | 15               | 0              | d <sub>3</sub>   | 0              |

The position of the wrist frame relative to the base frame was expressed as:

$$x = L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) = 110 \cos(\theta_1) + 90 \cos(\theta_1 + \theta_2)$$

$$y = L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) = 110 \sin(\theta_1) + 90 \sin(\theta_1 + \theta_2)$$

$$z = 50 - d_3$$

The overall transformation from the base frame to the tool frame was obtained by multiplying the homogeneous transformation matrices of each joint:

$$T_0^3 = \begin{bmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & 0 & L_1 \cos(\theta_1) + L_2 \cos(\theta_1 + \theta_2) \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 & L_1 \sin(\theta_1) + L_2 \sin(\theta_1 + \theta_2) \\ 0 & 0 & 1 & 50 - d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Where:

$$L_1 = 110\text{mm}$$

$$L_2 = 90\text{mm}$$

$$T_0^3 = \begin{bmatrix} \cos(\theta_1 + \theta_2) & -\sin(\theta_1 + \theta_2) & 0 & 110 \cos(\theta_1) + 90 \cos(\theta_1 + \theta_2) \\ \sin(\theta_1 + \theta_2) & \cos(\theta_1 + \theta_2) & 0 & 110 \sin(\theta_1) + 90 \sin(\theta_1 + \theta_2) \\ 0 & 0 & 1 & 50 - d_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

#### 4.1.5 Inverse Kinematics

Inverse kinematics was used to determine the joint variables required for the manipulator to reach a desired position in the task space. In this project, inverse kinematics was applied to map the intersections of the Go board into corresponding joint space configurations. For this purpose, only the first two joints were considered, as they fully defined the position in the  $xy$ -plane. The third joint did not influence planar motion and was therefore omitted in the analysis.

Let the desired wrist position be  $(x,y)$ . The radial distance from the base was:

$$r^2 = x^2 + y^2$$

Using the law of cosines, the second joint angle was given by:

$$\cos(\theta_2) = \frac{r^2 - L_1^2 - L_2^2}{2L_1L_2}$$
$$\theta_2 = \arccos\left(\frac{r^2 - L_1^2 - L_2^2}{2L_1L_2}\right)$$

The first joint angle was then obtained as:

$$\theta_1 = \text{atan2}(y, x) - \text{atan2}(L_2 \sin(\theta_2), L_1 + L_2 \cos(\theta_2))$$

Two possible solutions existed for  $\theta_1$  and  $\theta_2$ , corresponding to the elbow-up and elbow-down configurations of the manipulator. Both solutions were valid as they lied within the joint limits.

## 4.2 Hardware

### 4.2.1 Links CAD and 3d printing

To put all those theoretical calculations into practice and bring the SCARA robot to life, the system was designed to meet the defined requirements. The parts required to achieve the desired link lengths were designed using SolidWorks and manufactured using Bambu Studio on Bambu 3D printers. They were manufactured using PLA due to its ease of printing, low cost, and sufficient stiffness for the low-load application. The links were designed as lightweight rectangular beams [Figure 4] to provide sufficient stiffness while minimising mass. Thereby reducing the torque required at each joint, allowing for smaller and more efficient actuators. The masses of the individual links were determined to be 18g, 10g, and 1g for links 1, 2, and 3 respectively. All parts of the assembly model can be found in the appendix.

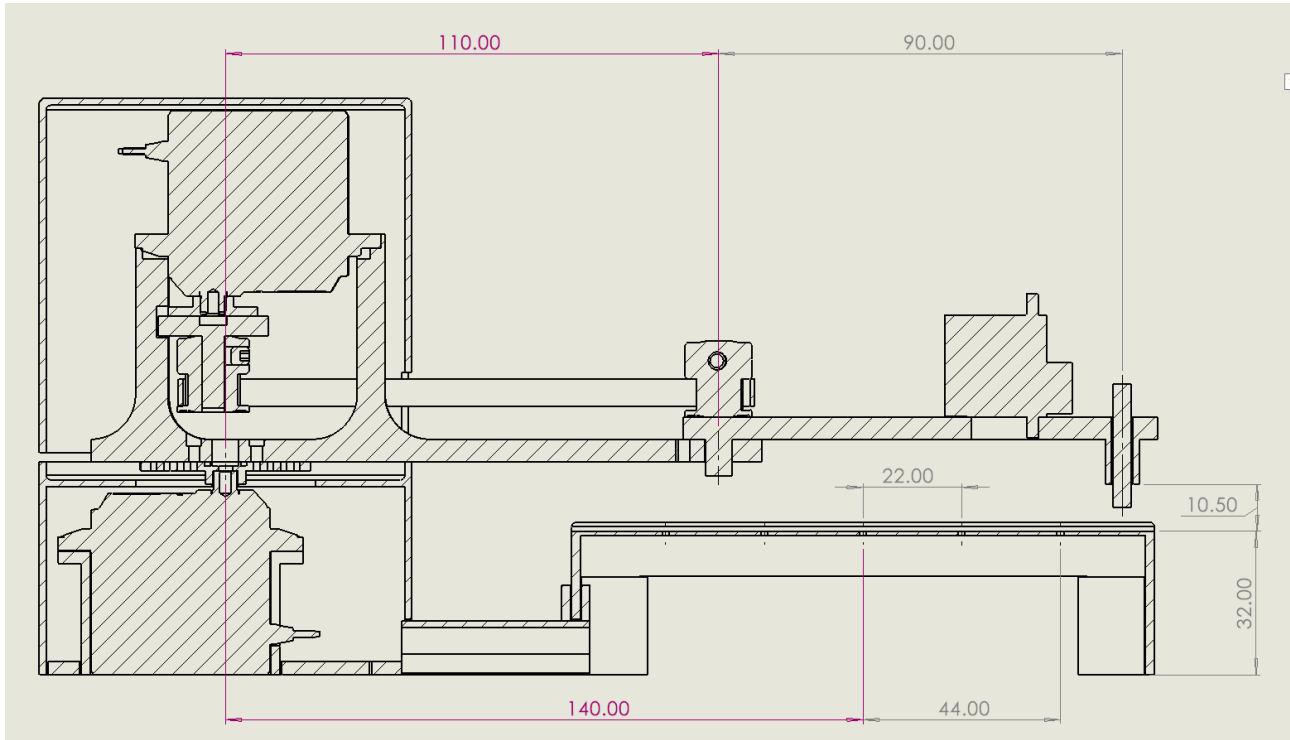


Figure 4: Assembled CAD Side view

#### 4.2.2 Actuation system

The selection of actuators was guided by the torque and range of motion required for the manipulator. For a robot joint, the required torque is the sum of the moments caused by all loads acting beyond that joint [equation 1].  $\tau$  is the torque and  $d$  represents the perpendicular distance from the joint axis.  $F$  is the sum of all the forces acting on an object, and  $g$  the gravitational force. In this case, the main force acting on the arm was the weight :

$$\tau = F * d \quad F = m * g \quad \tau = m * g * d$$

For joint 3, the mass of the combined end-effector and payload was negligible therefore only a small torque was required [Tab 4]. However, the same could not be said for joint 1. The static torque required at joint 1 was estimated from the masses of the links, end-effector, and payload, along with their distances from the joint axis. It was calculated as follows:

$$\tau_1 = m_1 * g * \frac{L_1}{2} + m_2 * g * \left( L_1 + \frac{L_2}{2} \right) + m_3 * g * (L_1 + L_2)$$

Where:

$m_1$ = mass of Link 1 and the motor at Joint 2 = 18 + 72 = 90 grams

$m_2$ = mass of Link 2 and the pulley at Joint 2 = 10 + 10 = 20 grams

$m_3$ = mass of Link 3, the servo at Joint 3, and the end-effector magnet = 1 + 9 + 2 = 12

$$\tau_1 = 90 * g * \frac{110}{2} + 20 * g * \left( 110 + \frac{90}{2} \right) + 12 * g * (110 + 90) = 1.05 \text{ kg} * \text{cm}$$

The selected actuator provided a stall torque of 11.5 kg\*cm. This provided a sufficient safety margin for reliable operation while accounting for friction and assembly tolerances. The same actuator was selected for joint 2, where the torque requirements were lower. Additionally, these motors were chosen to satisfy the desired operating range of 270 degrees.

Table 3. Actuator selection

| Model    | Type  | Weight | Stall Torque | Operating Voltage | Operating Range |
|----------|-------|--------|--------------|-------------------|-----------------|
| DMS-MG90 | Servo | 9g     | 1.8 kg / cm  | 4.8-6V            | 60° degrees     |
| DSS-M15S | Servo | 72g    | 13.5 kg / cm | 4.8-7.2V          | 270° degrees    |

#### 4.2.3 Transmission mechanism

To reduce the load acting on Joint 1, the actuator responsible for driving Joint 2 was positioned above Joint 1 rather than directly on Link 1. As a result, a transmission mechanism was required to transfer motion from the motor to joint 2 remotely. To achieve this, a timing belt and pulley system was implemented to the design. To maintain 1:1 transmission ratio and ensure that angular displacement was directly transferred from the motor to joint 2, identical GT2 models with a diameter of 12.22mm were used at both ends. This pulley profile was specifically designed to minimise backlash and provide accuracy. Finally, the centre distance between the two pulleys was set to 110 mm. Based on this configuration, the required belt length [eq 3] was calculated to ensure proper tension, reduce backlash and avoid poor transmission. This was particularly important given the precision requirements of the manipulator.

$$L = 2 * C + 1.57 * (D + d) + \frac{(D - d)^2}{4 * dC} = 258 \text{ mm}$$

Where:

$C$  = Centre to centre distance = 110 mm

$D = d$  = Diameter of pulley = 12.22 mm

$L$  = Length of timing belt

#### 4.2.4 End-Effector design

The end-effector featured both the Joint 3 and the gripping mechanism into a single unit. This design enabled the motor to perform both vertical motion and pick-and-place operations. The motor's angular motion was converted into vertical motion via a servo horn through a thin rod connected to link 3. In which lay within, at the bottom, a magnet that carried out the 'gripping' operation [Figure 5]. It was a passive system which only complied with light magnetic stones. The end effector operated in three distinct states. The low position [a] where the end-effector was brought into contact with the stone, allowing the magnet to attract and pick it up. The hovering position [b] where the end-effector was raised slightly above the board, enabling safe horizontal transport of the stone without interference with other pieces. And finally, the top position [c] where the end-effector was lifted further upward, causing the stone to lose contact with the magnet.

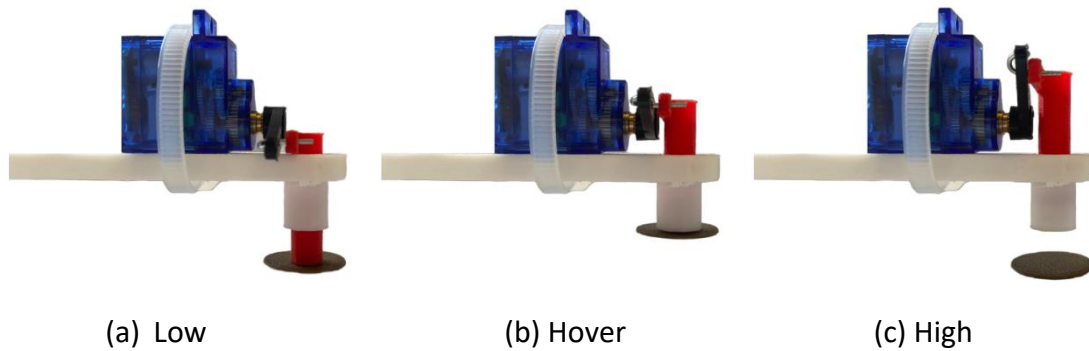


Figure 5: End-effector mechanism

#### 4.2.5 Power supply

The system was powered by four AA batteries, providing a total supply voltage of approximately 6V. This voltage was suitable for driving the selected servo motors, which operated within a range of 4.8 to 7.2 V. The system could easily be powered on and off with the switch provided on the battery container.

#### 4.2.6 Control Hardware

The servo motors were controlled by an Arduino Uno, which generated the required PWM signals to control the angular position of each joint. Each servo had 3 wires: power, ground and signal. The signal wire was connected to the Arduino via signal pins, while power and ground wires were supplied by the external power supply [Figure 6]. A common ground was maintained between the Arduino and the power supply to ensure proper signal reference. Lastly, the Arduino Uno was connected to a laptop via a USB cable, enabling the transmission of inputs from the playing interface.

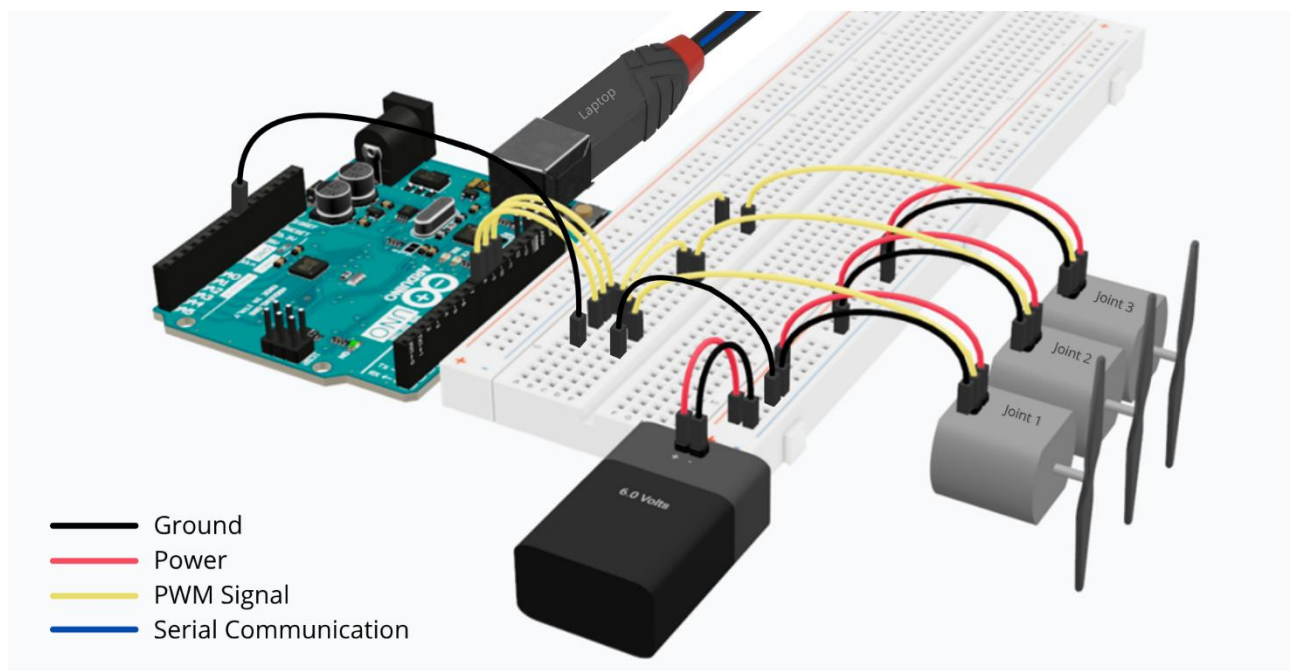


Figure 6: Circuit model

#### 4.2.7 Circuit and assembly

The electrical system of the device consisted of three servo motors, an Arduino microcontroller, and a dedicated power supply. To ensure a reliable and robust circuit, the permanent connections were soldered to a stripboard. This reduced the risk of loose connections. Cable management was also considered in the design to reduce clutter and improve ease of transport. Routing paths were incorporated into the CAD model to conceal and secure cables within the structure [Figure 7].

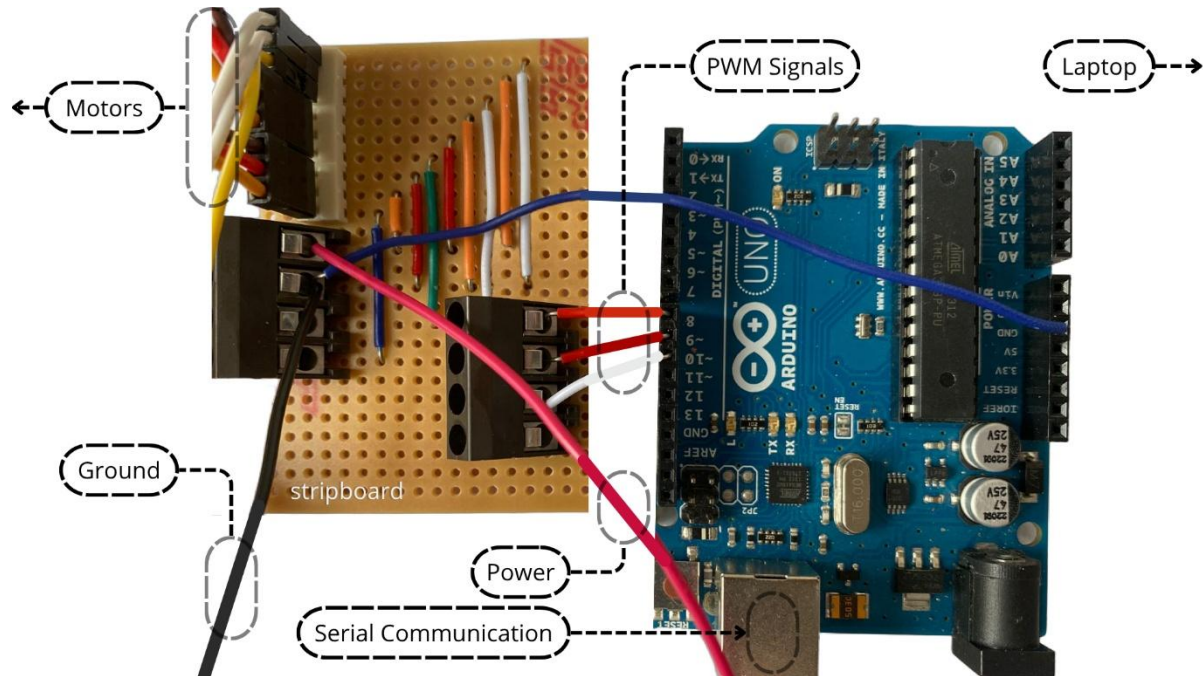


Figure 7: Circuit assembly

Finally, upon assembly, the components formed the complete SCARA robot system. The links, joints, and actuators were securely fastened using either cable ties, tight fits or screws and nuts. A casing was added to cover the components [Figure 8]. Although initially intended for visual appearance, the casing also contributed to the safety of the device.

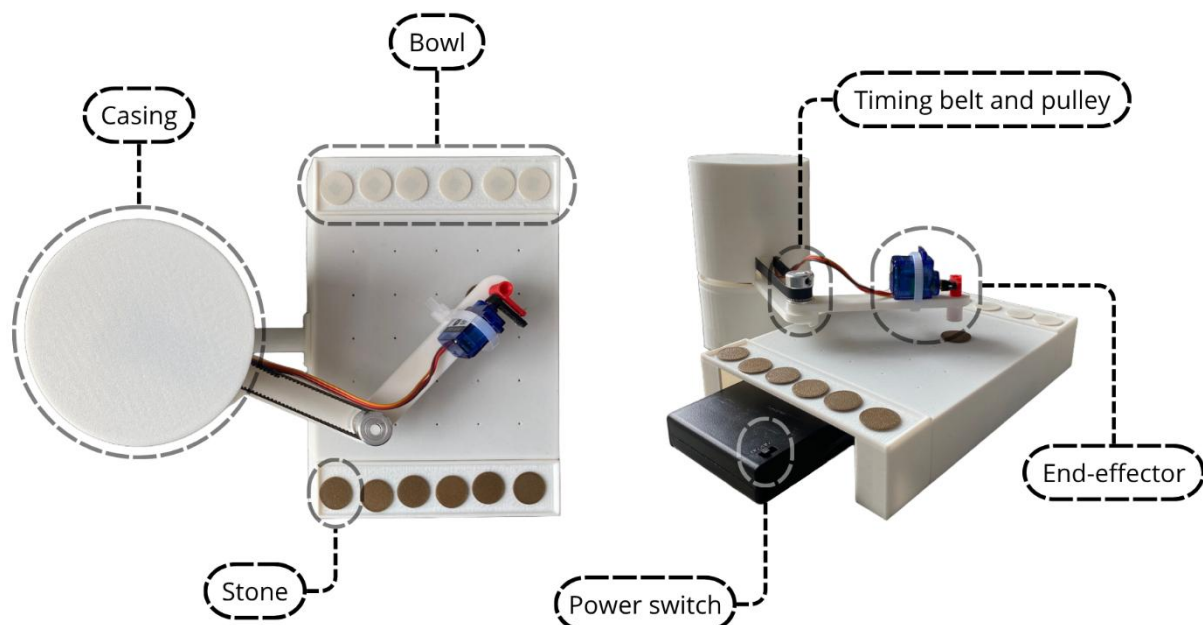


Figure 8: Go playing SCARA Robot

### 4.3 Software

#### 4.3.1 Arduino IDE

To control the three motors connected, the SCARA robot was programmed using Arduino IDE. This environment was used to write, compile and upload code to the Arduino board as well as send commands through the serial monitor [Figure 9].

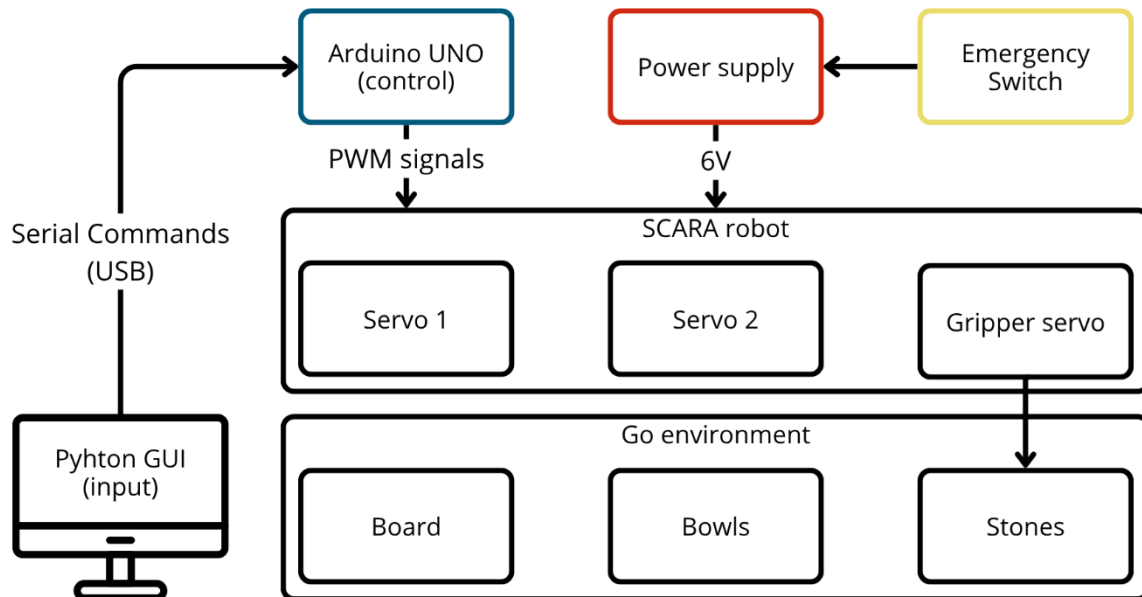


Figure 9: System Architecture

The project was separated into header files [Figure 10] to make the project easier to maintain and debug. In total, the folder consisted of seven files: the main Arduino *.ino* file and three pairs of *.cpp* (source) and *.h* (header) files. The *.cpp* files each contained a specific implementation, while the corresponding *.h* files declared functions and shared variables, making them accessible across different files. They can all be found in the appendix.

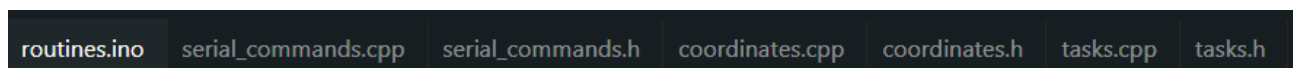


Figure 10: Project files

#### 4.3.2 System setup

The main file provided the foundation for the rest of the environment. An inverse kinematics model was implemented to enable the robotic arm to reach specific target positions on the board. Before this could be applied, the required libraries were imported to support the mathematical computations needed to determine joint angles from a given set of coordinates. As multiple joint configurations can exist for a single target position, a consistent configuration was selected. The physical dimensions of the arm and the offset between the board frame and the robot base were defined to allow accurate transformation of target coordinates into the arm's reference frame. The three servos controlling the joints and end-effector were initialised and assigned to specific Arduino pins [see Appendix], allowing the computed IK outputs to be translated into physical movement of the arm. In addition, serial communication was initialised in this file to allow

target positions to be sent from the laptop to the Arduino, to be processed by functions described in a later section.

#### 4.3.3 Target Coordinates

The coordinates file managed the representation and transformation of pre-defined target positions, supporting the inverse kinematics calculations. It included the description of the layout of the playing area. A predefined grid represented the board positions with 25 coordinates, while additional coordinate sets defined locations for the left and right bowls [Figure 11]. It also provided functions to convert user-friendly inputs into Cartesian board coordinates which are used by the inverse kinematics algorithm to generate the required joint movements.

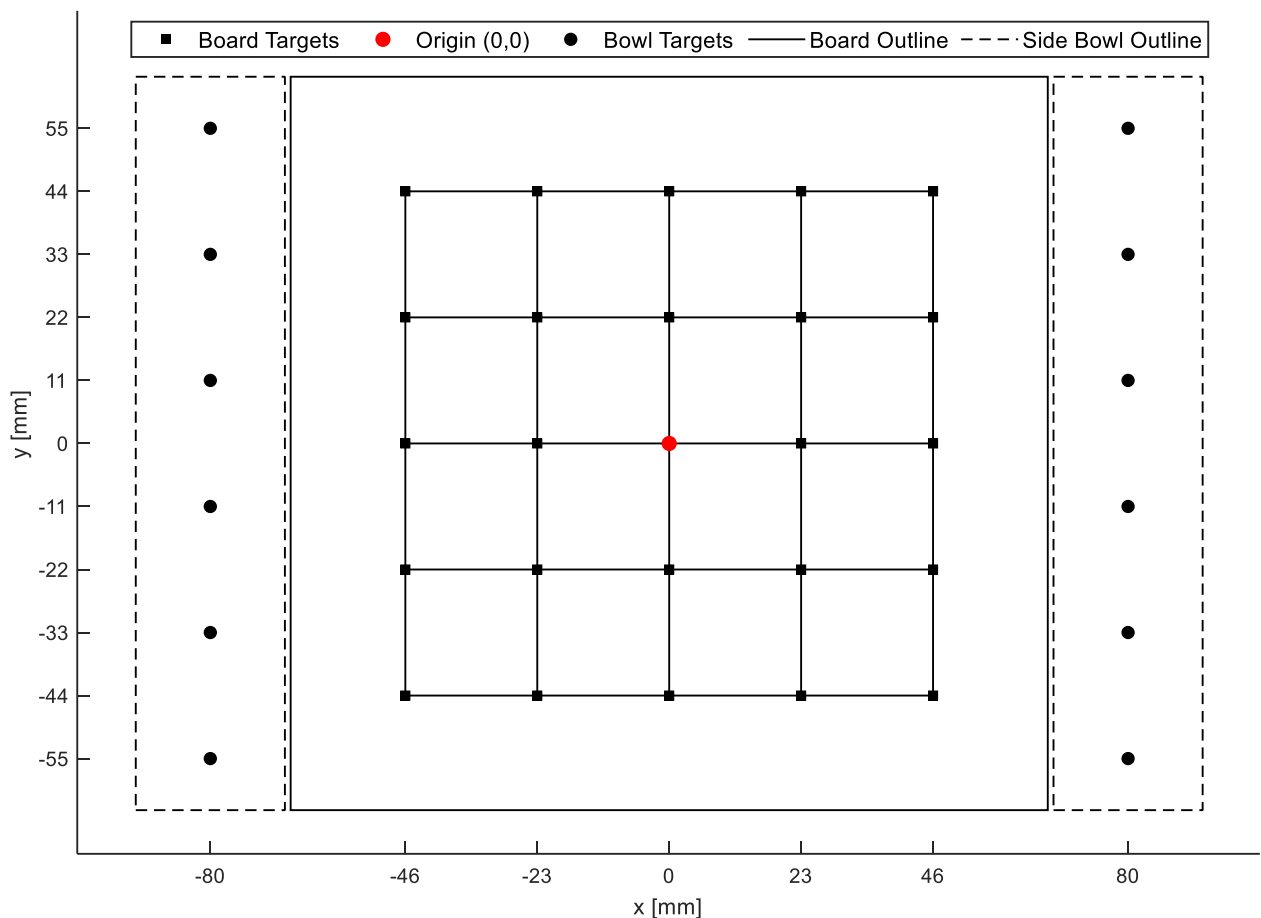


Figure 11: Cartesian coordinates of board and bowls

#### 4.3.4 Sequences

The tasks file defined higher-level behaviours, grouping sequences of movements into executable actions. The two highest-level routines of the robotic arm defined are placing or removing stones from the board [Figure 12]. This script enabled the conversion of simple inputs into complete task sequences, involving both the end-effector and the joints. It also tracked the state of the left and right bowls on either side of the board, by recording which slots contained stones and which ones were empty. This allowed the system to automatically select valid pickup and drop-off locations during operation. While this file handled main routines, the following file took charge of the simpler ones which did not require consecutive steps.

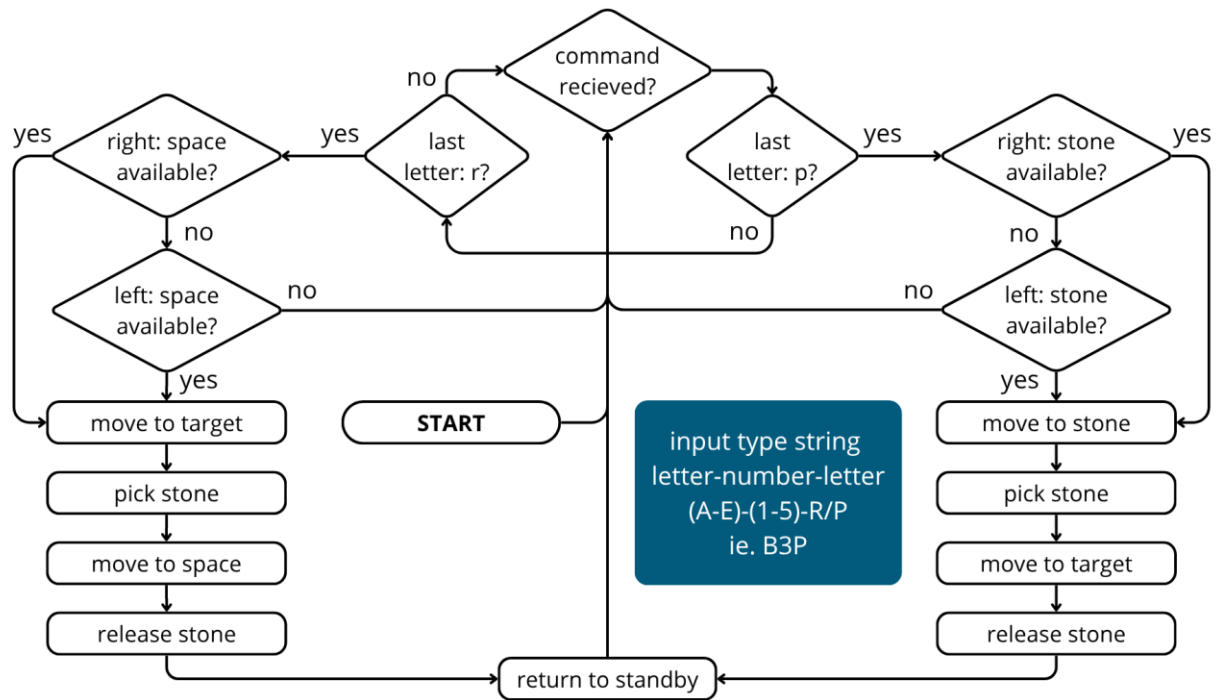


Figure 12: Arduino logic diagram

#### 4.3.5 Serial commands

The commands file interpreted incoming commands from the serial monitor in real time. This script was essential as it acted as the middleman between the user input and the motion functions of the robotic arm. Not only did it read incoming serial data, interpreted the command format, and called the corresponding function to handle the data. It also, supported direct movement commands, predefined arm positions, end-effector controls, and the simple pick-and-place tasks. Moreover, it distinguished between board coordinates and storage box locations, allowing different parts of the workspace to be addressed through a simple command syntax. Input validation was included to reject invalid commands and provide feedback through the serial monitor.

#### 4.3.6 Playing Interface

A graphical user interface [Figure 13] was initialised in Python using the Tkinter library to provide a way of controlling the robotic arm. This interface was used to effectively convert user inputs into commands that are interpreted by the Arduino system. To do so, the interface displayed an interactive 5x5 Go board, allowing the user to place or remove stones by clicking on board intersections, with either a right or left using any standard mouse or keypad. Each interaction updated the board display and generated a corresponding command, formatted as shown on Figure 13. Within the command string, the first character represented the column of the target on the board, the second represented the row, and the third indicated the action, either 'P' or 'R', corresponding to place or remove as defined in the Arduino system. Eventually, these commands were then immediately passed to the Arduino via serial communication.

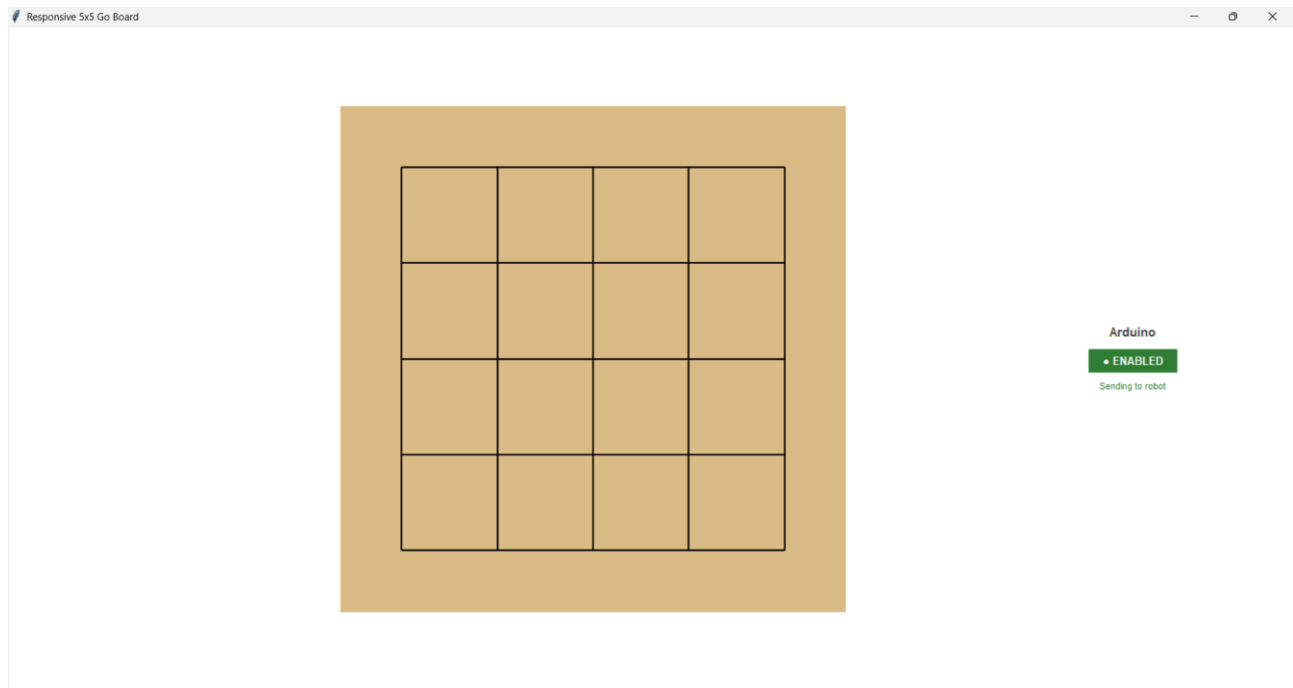


Figure 13: Graphical user interface display

#### 4.3.7 Workflow

The process was divided into three main stages: user interface, control system processing, and actuation [Figure 14]. The workflow began with the user's input through the Python-based graphical user interface. The player selected an intersection on the board. The selected target position was formatted into a command before being transmitted to the Arduino via serial communication. Upon receiving the command, the content was parsed. The Arduino determined the required action, such as placing or removing a stone. The selected target was then converted into Cartesian coordinates through coordinate mapping. These coordinates were passed to the inverse kinematics algorithm, which computed the required joint angles for the robotic arm to reach the desired position. Then the Arduino passed the corresponding PWM signals to the servo motors. Each servo executed the angular motion accordingly until the task was completed and the user satisfied.

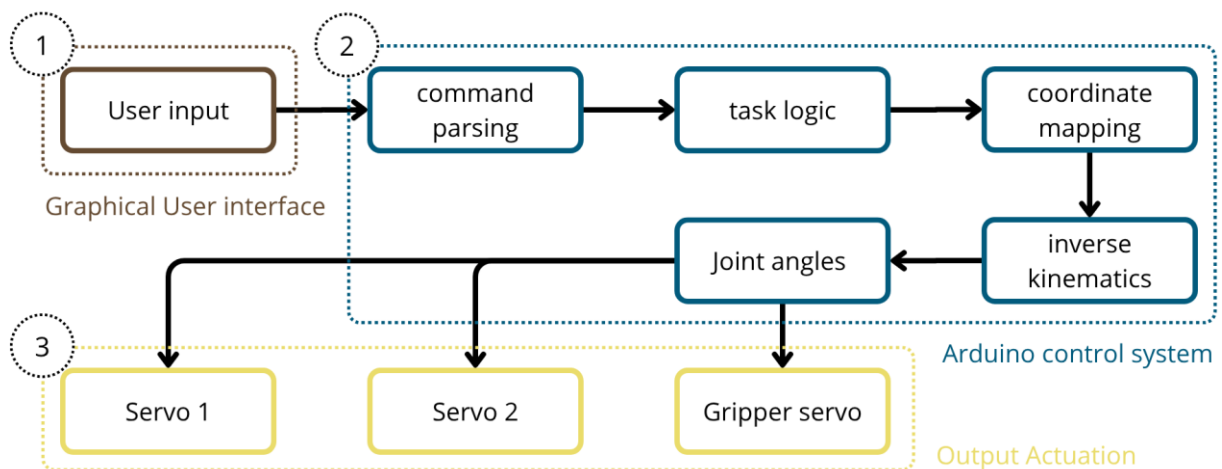


Figure 14: System workflow

## 5 Conclusion

### 5.1 Results

This report set out to design and implement a robotic system capable of autonomously performing pick-and-place operations on a 5×5 Go board. The objectives were successfully achieved with the integration of mechanical design, control software, and user interface.

A complete 3-DOF robotic arm was successfully designed and fabricated, including the CAD model, actuator selection, and mechanical assembly. The system operated as intended, with all joints functioning and capable of reaching the defined workspace. A functional gripper mechanism was developed and integrated with the arm. The gripper was able to pick, and place Go stones effectively using a magnetic approach with 100% success rate. Forward and inverse kinematics were successfully implemented, enabling conversion between joint space and end-effector position. The inverse kinematics solver reliably generated valid joint angles for reachable targets, while rejecting unreachable positions, contributing to overall system robustness. A command interface was implemented to accept board coordinates and actions (pick/place), which were transmitted via serial communication from the Python interface to the Arduino. These commands were successfully interpreted and converted into joint-space movements, enabling real-time control of the robotic arm.

Additionally, an emergency stop provided by the power supply switch allowed the system to be safely halted at any time by cutting power to the servos, further providing safety measures. In terms of performance, the system demonstrated good precision and repeatability, consistently returning to the same positions when executing identical commands. However, a slight positional error was observed at joint 2 during anticlockwise motion, which occasionally resulted in failed pickups (when the stone was outside the magnetic range) or slight misplacement, causing stones to overlap. The system showed generally good reliability, although an abrupt movement was observed each time the system was initialised with servos jumping to their zero positions. This undesirable behaviour was not managed and presents a potential safety concern. Finally, while the speed was slow, the system achieved satisfactory responsiveness, with smooth motion.

### 5.2 Limitations

The results of this project highlighted several limitations. The robotic arm operated within a limited workspace, constrained by the link dimensions and the range of motion of its joints. As it were, the uppermost locations in the storage bowls were not reachable. This limitation could have been addressed through redesign, for example by repositioning the bowls closer to the arm or modifying link lengths. Additionally, the system had restricted vertical motion. These constraints affected the system's applicability to real-world scenarios. For instance, if used with a standard Go set, the current design would be unsuitable, as the stones are much taller. The gripper relied on magnetic attraction, whereas real Go stones are typically non-magnetic, making reliable pickup infeasible without redesigning the end-effector.

A key source of inaccuracy was identified at Joint 2, where slight positional errors were observed due to backlash in the transmission system. This backlash was likely due to the use of PLA which suffered deformation under tension. This deformation reduced belt tension by slightly decreasing

the distance between pulleys, increasing slack and therefore amplifying backlash. This issue could have been mitigated through improved mechanical design. However, its impact was exacerbated by the fact that the system operated in an open-loop configuration. As the current design does not incorporate any form of sensor-based feedback, positioning accuracy depended entirely on repeatable mechanical behaviour. Consequently, the system was unable to detect nor correct inaccuracies during operation.

### 5.3 Discussion

This project established a practical and structured framework for a Go-playing robot. Therefore, a natural progression of this work would be to extend the system towards full autonomy, independent of the user input. Implementing a computer vision system would be a fruitful area for further work. Enabling the robot to detect the state of the board in real time and thereby dynamically adapt to changes in the environment. Further work should also be carried out to integrate a pre-existing Go AI engine to enhance the capabilities of the system. By combining perception with decision-making, the robot could autonomously play by selecting optimal moves based on the current board state provided by the vision system. It is safe to say that considerably more work needs to be done until an autonomous robot that can master the ancient game of Go can be achieved.

## 6 Acknowledgements

The author would like to thank their supervisor, Massimiliano Zecca, for their guidance, support, and valuable feedback throughout the duration of this project. The author would also like to thank the laboratory technician, David King, for his assistance in providing components and access to laboratory facilities for experimental work. Appreciation is extended to Brijesh Gurav for manufacturing the components using the 3D printing facilities at Wolfson School, and to David Wayment for coordinating the ordering of parts and providing updates on their delivery. Last but not least, the author would like to thank Emma Taylor for her invaluable feedback and advice on the writing of this report.

## 7 References

- [1] "Informatical Analysis of GO, Part 1: Evolutionary changes of board size," *IEEE Conference Publication / IEEE Xplore*, Aug. 01, 2020. <https://ieeexplore.ieee.org/abstract/document/9231758>
- [2] H. Matsubara, H. Iida, and R. Grimbergen, "Natural developments in game research," *ICGA Journal*, vol. 19, no. 2, pp. 103–112, Jun. 1996, doi: 10.3233/icg-1996-19208.
- [3] M. Sousa and E. Bernardo, "Back in the game," in *Communications in computer and information science*, 2019, pp. 72–85. doi: 10.1007/978-3-030-37983-4\_6.
- [4] M. J. Rogerson, M. Gibbs, and W. Smith, "'I Love All the Bits,'" *ACM Digital Library*, pp. 3956–3969, May 2016, doi: 10.1145/2858036.2858433.
- [5] Sensetime, "SenseTime Unveils SenseRobot Go: The Second AI-Powered Board Game Robot Joins the SenseRobot Family-News and Stories-SenseTime," *Sensetime.com*, 2023. <https://www.sensetime.com/en/news-detail/51166774?categoryId=1072>

- [6] P. K. Rath, N. Mahapatro, P. Nath, and R. Dash, "Autonomous Chess Playing Robot," *IEEE Xplore*, Oct. 01, 2019. <https://ieeexplore.ieee.org/document/8956389>
- [7] Pawel Kolosowski, A. Wolniakowski, and Kanstantsin Miatliuk, "Collaborative Robot System for Playing Chess," Jul. 2020, doi: <https://doi.org/10.1109/msm49833.2020.9202398>.
- [8] D. Springwald, "iGoBot - Daniel Springwalds Blog \* Basteln mit Daniel," *Springwald.de*, 2018. <https://blog.springwald.de/post/2018/iGoBot>
- [9] C. Matuszek *et al.*, "Gambit: An autonomous chess-playing robotic system," *IEEE Xplore*, pp. 4291–4297, May 2011, doi: 10.1109/icra.2011.5980528.
- [10] T. T. Mac, T.-N. Dam, T. Tri, Ly Hoang Hiep, and T.-D. Nguyen, "The Development of an Intelligent Robot Arm Playing chess with Human-Robot Interaction," *IEEE Conference Publication / IEEE Xplore*, Dec. 2023, doi: <https://doi.org/10.1109/ichst59286.2023.10565356>.
- [11] K.-L. Su, "Implementation of the chess game artificial intelligent using mobile robots," *IEEE Conference Publication / IEEE Xplore*, Dec. 01, 2014. <https://ieeexplore.ieee.org/document/7044846> (accessed Apr. 09, 2026).
- [12] D. Silver *et al.*, "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, no. 7587, pp. 484–489, Jan. 2016, doi: <https://doi.org/10.1038/nature16961>.
- [13] D. J. Wu, "Accelerating Self-Play Learning in Go," *arXiv.org*, Feb. 27, 2019. <https://arxiv.org/abs/1902.10565>
- [14] S. Surati, S. Hedao, T. Rotti, V. Ahuja, and N. Patel, "Pick and Place Robotic Arm: A Review Paper." Accessed: Apr. 20, 2026. [Online]. Available: <https://www.irjet.net/archives/V8/i2/IRJET-V8I2311.pdf>
- [15] A. Antonov, "Parallel–Serial Robotic Manipulators: A Review of Architectures, Applications, and Methods of Design and Analysis," *Machines*, vol. 12, no. 11, p. 811, Nov. 2024, doi: <https://doi.org/10.3390/machines12110811>.
- [16] Yogeshwarappa.T, Yogeshwarappa.T, Yogeshwarappa.T, and Yogeshwarappa.T, "AN OVERVIEW OF ROBOTICS AND AUTOMATION IN MODERN INDUSTRY," *IJCRT - International Journal of Creative Research Thoughts (IJCRT)*, vol. 8, no. 12, pp. 3609–3619, Dec. 2020, Accessed: Apr. 20, 2026. [Online]. Available: [https://ijcrt.org/viewfull.php?p\\_id=IJCRT2012408](https://ijcrt.org/viewfull.php?p_id=IJCRT2012408)
- [17] S. Kaewkorn, C. Joochim, P. Keeratiwintakorn, and A. Kunapinun, "Development of Pick and Place Delta Robot," *Advances in Intelligent Systems and Computing*, pp. 475–486, doi: 10.1007/978-3-030-40271-6\_47.
- [18] P. Kushwaha, J. Kumar, D. Ajit, A. Chauhan, N. Jain, and A. Verma, "Design and Fabrication of Industrial Pick and Place Delta Robot," in *Unraveling New Frontiers and Advances in Bioinformatics*, 2024, pp. 409–423. doi: 10.1007/978-981-97-7123-3\_18.
- [19] A. elassal, M. abdelal, Mohmad Osama, and Hager elhnidy, "low-cost parallel delta robot for a pick-and-place application with the support of the vision system.," *e-Prime, advances in electrical engineering, electronics and energy*, pp. 100518–100518, Mar. 2024, doi: <https://doi.org/10.1016/j.prime.2024.100518>.

- [20] S. Thatere, P. Arora, Y. Sharma, R. Kumar, and R. G. Mishra, "Design and Implementation of a Gantry Robot for Pick and Place Mechanism with Obstacle Detection using Programmable Logic Controller," *Zenodo*, Jun. 2016, doi: <https://doi.org/10.5281/zenodo.161331>.
- [21] G. S. Sharath, N. Hiremath, and G. Manjunatha, "Design and analysis of gantry robot for pick and place mechanism with Arduino Mega 2560 microcontroller and processed using pythons," *Materials Today: Proceedings*, vol. 45, pp. 377–384, 2021, doi: <https://doi.org/10.1016/j.matpr.2020.11.965>.
- [22] "DEVELOPMENT OF A CARTESIAN ROBOT WITH IMAGE PROCESSING AND GRASP DETECTION - SMART 2023 - EASD Procedia," *Eccomas Procedia*, 2023. <https://www.eccomasproceedia.org/conferences/thematic-conferences/smart-2023/9872> (accessed Apr. 20, 2026).
- [23] S. F. Noshahi, A. Farooq, M. Irfan, T. Ansar, and N. Chumuang, "Design and Fabrication of an Affordable SCARA 4-DOF Robotic Manipulator for Pick and Place Objects," *2019 14th International Joint Symposium on Artificial Intelligence and Natural Language Processing (iSAI-NLP)*, Oct. 2019, doi: <https://doi.org/10.1109/isai-nlp48611.2019.9045203>.
- [24] W.-B. Li, G.-Z. Cao, X.-Q. Guo, and S.-D. Huang, "Development of a 4-DOF SCARA robot with 3R1P for pick-and-place tasks," *IEEE Xplore*, Dec. 01, 2015. <https://ieeexplore.ieee.org/document/7398909> (accessed Nov. 17, 2021).
- [25] M. A. González-Palacios, M. A. Garcia-Murillo, and M. González-Dávila, "A novel tool to optimize the performance of SCARA robots used in pick and place operations," *Journal of Mechanical Science and Technology*, vol. 35, no. 10, pp. 4715–4726, Sep. 2021, doi: <https://doi.org/10.1007/s12206-021-0937-x>.

## 8 Appendix

Table 4. Bill of materials

| Item         | Quantity | Cost    | Supplier      | Reference |
|--------------|----------|---------|---------------|-----------|
| battery box  | 1        | £ 2,41  | RS components |           |
| Batteries    | 8        | £ 4,39  | RS components |           |
| Servo motors | 2        | £ 22,02 | Farnell       | SER0044   |
| Micro servo  | 1        | £ 4,32  | Farnell       | SER0039   |

Arduino Cloud Sketch

<https://app.arduino.cc/sketches/9923f9a1-adf1-4cbb-a925-ab8f35f5f6f3?view-mode=preview>

Project GitHub Repository

<https://github.com/umbelinapia/GoPlayingRobot>